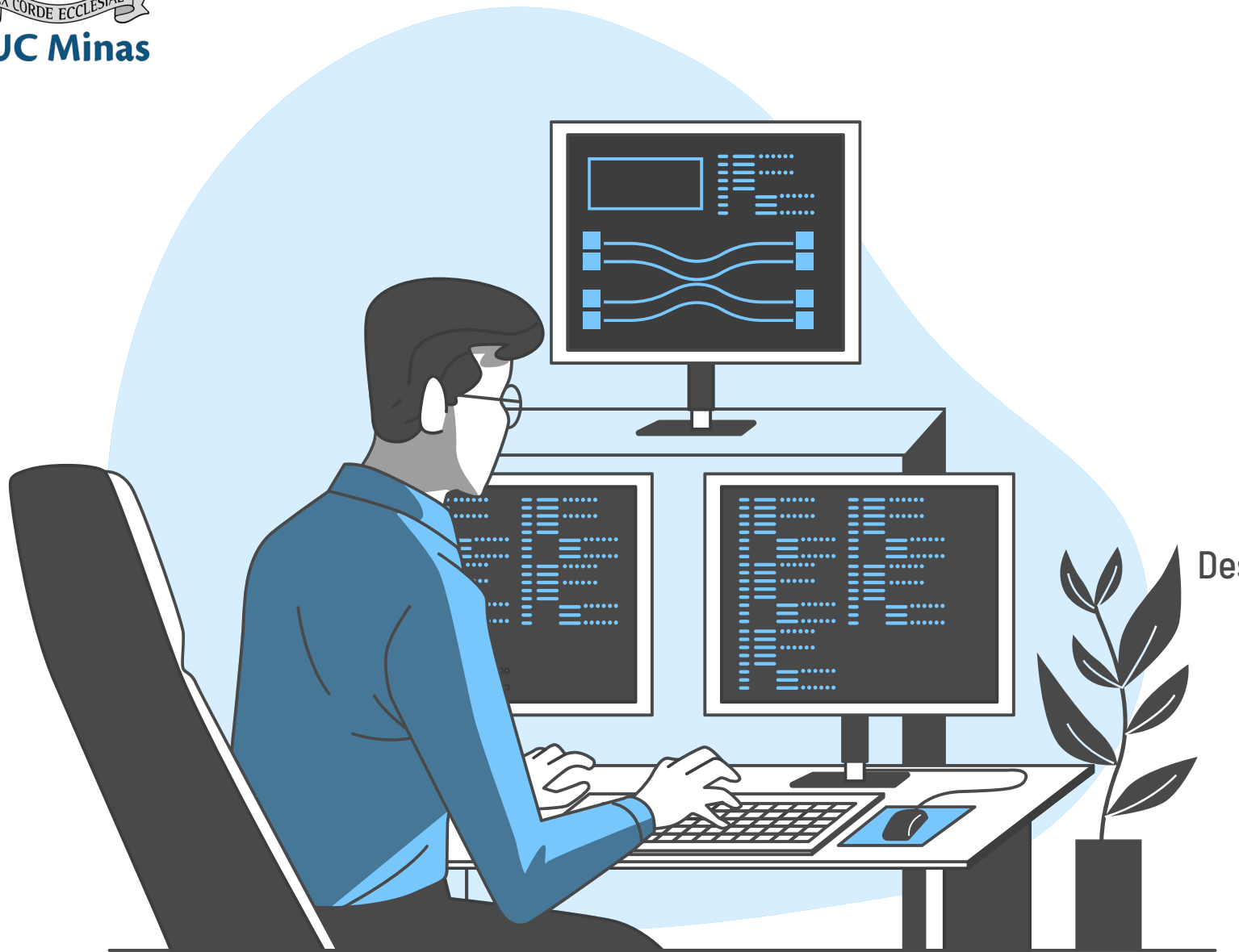
PUC Minas



Aula 03 · Back-end

pom.xml e application.properties

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1

Ler e editar o `pom.xml`: *parent*, coordenadas, `java.version`, dependências e *plugins*

2

Entender *starters*, `scope` e por que quase nenhuma dependência tem versão

3

Configurar o `application.properties`: porta, banco, CORS e propriedades próprias

4

Usar variáveis de ambiente com `${VARIAVEL:padrao}` e *profiles*

5

Ler propriedades no código com `@Value` e `@ConfigurationProperties`

Apresentação da aula

Antes de começar

- Aula 02: projeto Spring Boot criado

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 Parte 1: o pom.xml

02 Parte 2: o application.properties

03 Lendo propriedades no código

04 Profiles: configurações por ambiente

05 E o .gitignore?

06 Aprofundando: dependências e configuração externa

01

Parte 1: o pom.xml

Parte 1: o pom.xml

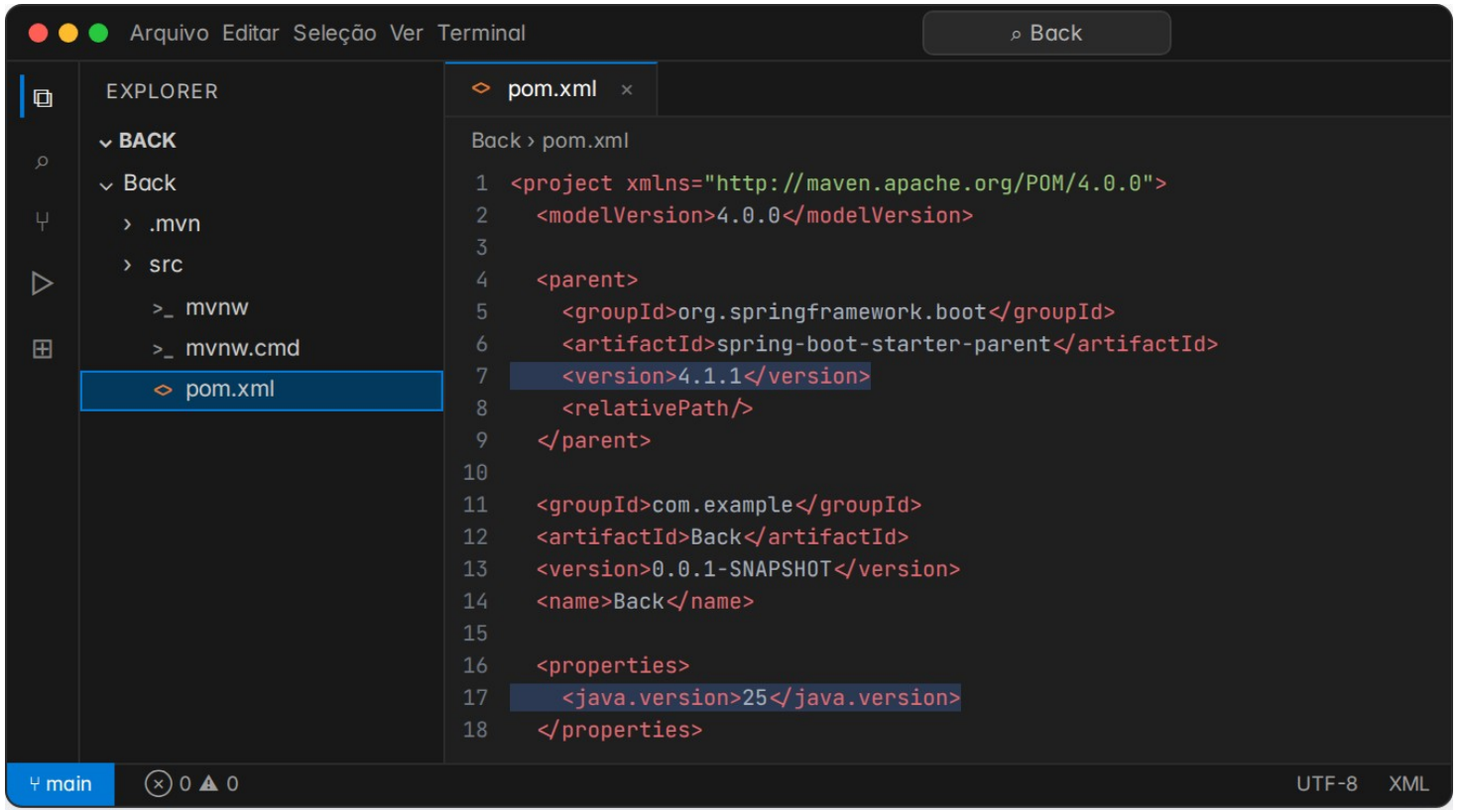
POM significa *Project Object Model*. É um arquivo XML na raiz do projeto que o Maven lê para saber quem é o projeto, de quais bibliotecas ele depende e como deve ser construído. Vamos ler o `pom.xml` do projeto `Exemplo_Integracao/Back`, ajustado para Java 25:

01

Parte 1: o pom.xml

Aula 03 · Back-end

Parte 1: o pom.xml



Início do `pom.xml`: o *parent* do Spring Boot, as coordenadas do projeto e a versão do Java.

Parte 1: o pom.xml

Bloco	Significado
``	Herda as configurações do Spring Boot. É ele que define a **versão de todas as dependências** compatíveis entre si. Para atualizar o Boot, troque só este número.
`groupId`, `artifactId`, `version`	As **coordenadas** do seu projeto. `SNAPSHOT` indica versão em desenvolvimento.
``	Versão do Java usada para compilar. Deve ser **25** e bater com o JDK instalado e com a imagem Docker.

Dependências

pom.xml (trecho de dependências)

```xml
<dependencies>
  <!-- API REST + Tomcat embutido (no Boot 3 o nome era spring-boot-starter-web) -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
  </dependency>

  <!-- Banco relacional (Supabase): JPA + driver do PostgreSQL -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
```

Dependências (cont.)

pom.xml (trecho de dependências)

```
  </dependency>

  <!-- Biblioteca de fora do Spring: a versão precisa ser informada -->
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.13.0</version>
  </dependency>

  <!-- Só existe durante os testes -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Dependências

Repare: as dependências do Spring **não têm ``**. O *parent* já sabe qual versão de cada uma funciona com o Boot 4.1.1. Só bibliotecas externas (como o JJWT) precisam da versão.

scope	Disponível em	Exemplo
(padrão) `compile`	Compilação, testes e execução	starters
`runtime`	Só na execução (o código não importa nada dela)	driver do PostgreSQL
`test`	Só nos testes	starters de teste
`provided`	Compilação; não entra no `.jar`	Lombok

Build e plugins

pom.xml (build)

```
<build>
  <finalName>app</finalName>  <!-- opcional: gera target/app.jar em vez de Back-0.0.1-SNAPSHOT.jar
-->
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

Build e plugins

O `spring-boot-maven-plugin` é quem cria o **fat jar**: um único `.jar` com seu código, todas as dependências e o Tomcat dentro. É ele também que fornece o comando `spring-boot:run`.

01

Parte 1: o pom.xml

Aula 03 · Back-end

Build e plugins

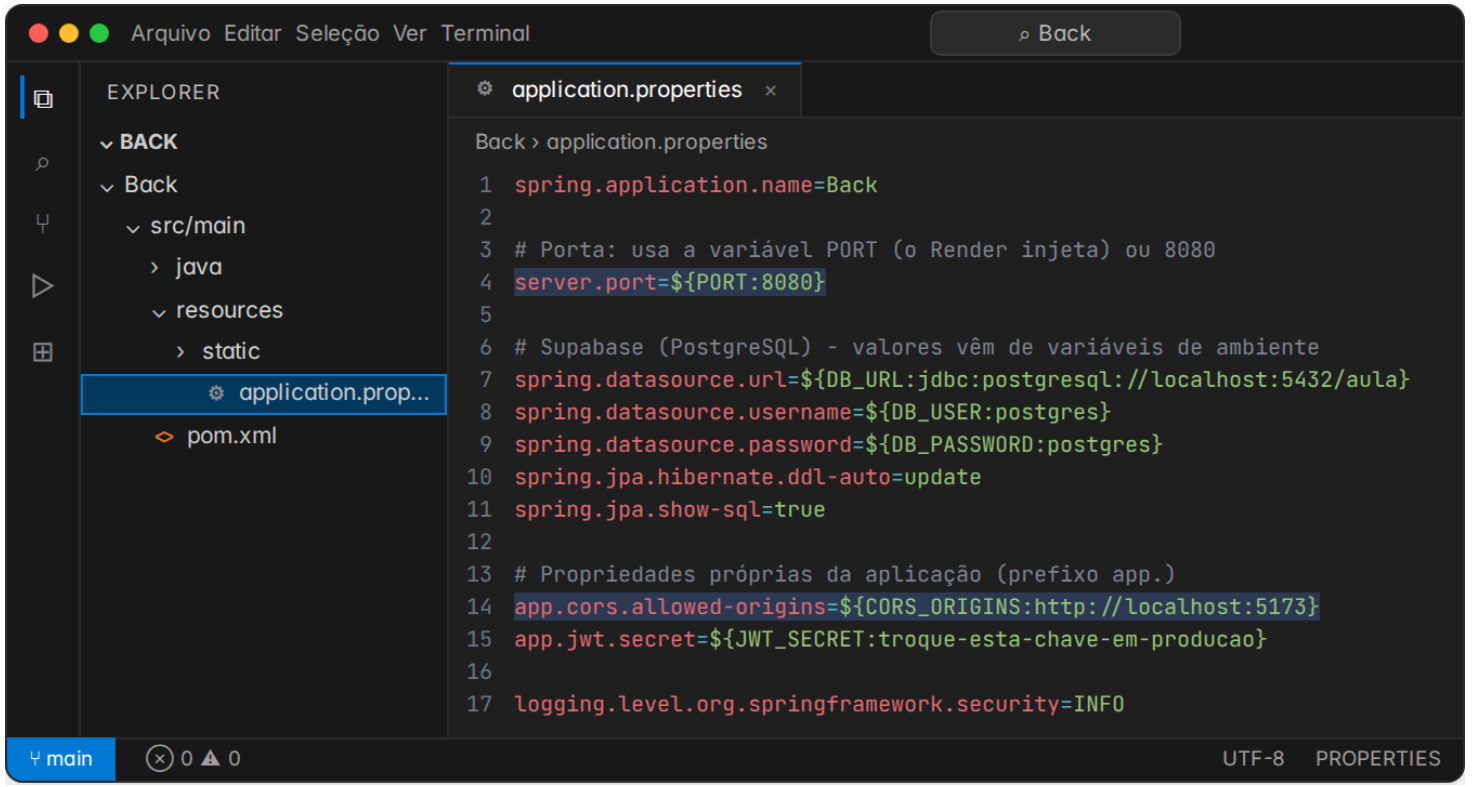
Dica

Adicionou uma dependência no `pom.xml` e o VS Code continua sublinhando os `import` em vermelho? Salve o arquivo e aceite o aviso **"A build file was modified. Do you want to synchronize...?"** (ou `Ctrl+Shift+P` → **Java: Reload Projects**). Pelo terminal, `./mvnw clean compile` também resolve.

02

Parte 2: o application.properties

Parte 2: o application.properties



Fica em `src/main/resources` e controla o comportamento da aplicação **sem mudar código**: porta, endereço do banco, nível de log, CORS. Cada linha é `chave=valor`; `#` inicia um comentário.

`application.properties` com propriedades do Spring e propriedades próprias (`app.*`).

A sintaxe ${VARIAVEL:padrao}

É o recurso mais importante para o deploy. O Spring procura uma **variável de ambiente** com aquele nome; se não existir, usa o valor depois dos dois-pontos. Assim, o mesmo arquivo funciona:

- **no seu computador**: nenhuma variável definida, valem os padrões (`8080`, `localhost`);
- **no Render**: você cadastra `PORT`, `DB_URL`, `DB_PASSWORD` no painel e o Spring usa esses valores.

02

Parte 2: o application.properties

Aula 03 · Back-end

A sintaxe ${VARIAVEL:padrao}

Atenção

Nunca escreva senhas reais do Supabase ou do MongoDB Atlas como valor padrão em um repositório público. Robôs varrem o GitHub procurando *connection strings*. Deixe no arquivo só valores de desenvolvimento e coloque os reais em variáveis de ambiente.

Propriedades mais usadas

Propriedade	Para que serve
`server.port`	Porta HTTP (padrão 8080)
`spring.datasource.url`, `.username`, `.password`	Conexão com banco relacional (Supabase)
`spring.jpa.hibernate.ddl-auto`	`update` cria/altera tabelas a partir das entidades; `none` não mexe; `create-drop` recria a cada execução
`spring.jpa.show-sql`	Mostra no console o SQL gerado
`spring.mongodb.uri`	Conexão com o MongoDB (no Boot 3 era `spring.data.mongodb.uri`)

Propriedades mais usadas (cont.)

Propriedade	Para que serve
`spring.thymeleaf.cache`	`false` em desenvolvimento para ver alterações no HTML
`server.servlet.context-path`	Prefixo para todas as URLs (ex.: `/api`)
`spring.servlet.multipart.max-file-size`	Tamanho máximo de upload
`logging.level.`	Nível de log (`DEBUG`, `INFO`, `WARN`...)

03

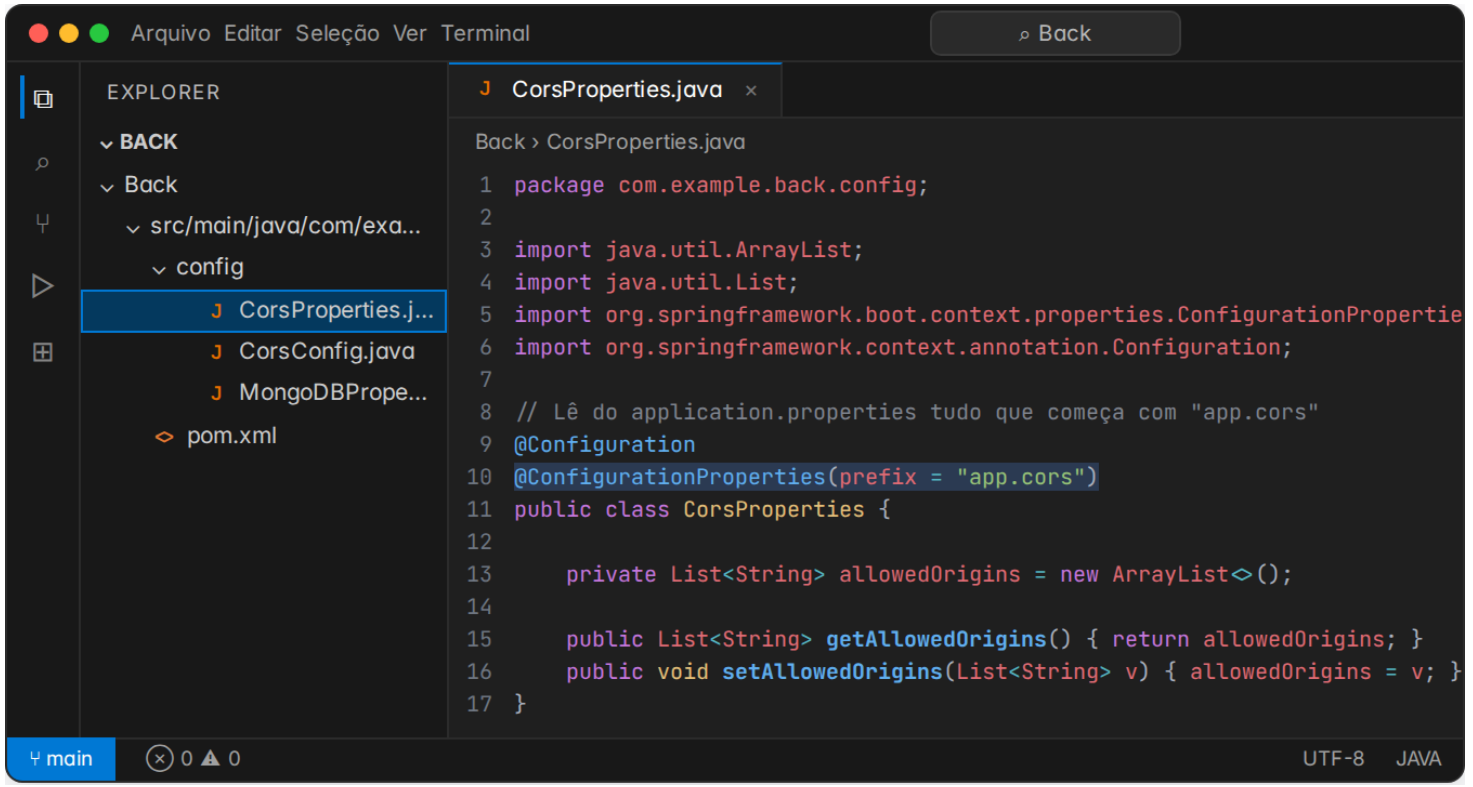
Lendo propriedades no código

Opção 1: @Value (um valor só)

```
@Value
```

```java
@Service
public class TokenService {
    @Value("${app.jwt.secret}")
    private String segredo;
}
```

Opção 2: @ConfigurationProperties (um grupo de valores)



É o padrão usado nos projetos da disciplina, no pacote `config`. O Spring lê tudo que começa com um prefixo e preenche os atributos de uma classe, convertendo `allowed-origins` em `allowedOrigins` e separando listas pela vírgula.

Classe de propriedades no pacote `config`, igual à do projeto `Exemplo_Integracao`.

04

Profiles: configurações por ambiente

Profiles: configurações por ambiente

Você pode ter arquivos extras como `application-dev.properties` e `application-prod.properties`. O arquivo principal vale sempre; o do *profile* ativo sobrescreve as chaves repetidas.

Profiles

```
# application.properties
spring.profiles.active=${SPRING_PROFILES_ACTIVE:dev}

# application-dev.properties
spring.jpa.show-sql=true
logging.level.com.example=DEBUG

# application-prod.properties
spring.jpa.show-sql=false
spring.jpa.hibernate.ddl-auto=validate
```

Profiles: configurações por ambiente

Observação

O Spring também aceita o formato **YAML** (`application.yml`), com indentação no lugar dos pontos. É só outra forma de escrever a mesma coisa. Nos projetos usamos `.properties`, mas os dois funcionam; não mantenha os dois ao mesmo tempo para não se confundir.

05

E o .gitignore?

E o .gitignore?

O projeto gerado já vem com um `.gitignore` que exclui `target/` e arquivos de IDE. Se você decidir guardar segredos em um arquivo separado (por exemplo `application-local.properties` ou `.env`), acrescente-o ao `.gitignore`:

.gitignore

```
target/
.idea/
*.iml
.vscode/
application-local.properties
.env
```

06

Aprofundando: dependências e configuração externa

Aprofundando: dependências e configuração externa

Árvore de dependências

```
$ ./mvnw dependency:tree
[INFO] com.example:Back:jar:0.0.1-SNAPSHOT
[INFO] +- org.springframework.boot:spring-boot-starter-webmvc:jar:4.1.1:compile
[INFO] |  +- org.springframework.boot:spring-boot-starter-jackson:jar:4.1.1:compile
[INFO] |  +- org.springframework.boot:spring-boot-starter-tomcat:jar:4.1.1:compile
[INFO] |  |  \- org.apache.tomcat.embed:tomcat-embed-core:jar:11.0.13:compile
[INFO] |  \- org.springframework:spring-webmvc:jar:7.0.1:compile
[INFO] +- org.springframework.boot:spring-boot-starter-data-mongodb:jar:4.1.1:compile
[INFO] |  \- org.mongodb:mongodb-driver-sync:jar:5.6.1:compile
```

Aprofundando: dependências e configuração externa

Um *starter* puxa dezenas de dependências **transitivas**. O `dependency:tree` mostra quem trouxe cada biblioteca, o que ajuda a entender conflitos de versão. As versões exatas da saída acima são ilustrativas e mudam a cada versão do Boot.

06

Aprofundando: dependências e configuração externa

Aula 03 · Back-end

Aprofundando: dependências e configuração externa

Atenção

Evite fixar `` em dependências que o Spring Boot já gerencia. Se você escrever uma versão diferente da que o *parent* escolheu, pode quebrar a compatibilidade entre bibliotecas. Só informe versão em bibliotecas externas (JJWT, springdoc, frontend-maven-plugin).

Aprofundando: dependências e configuração externa

Ordem de precedência (maior vence)	Exemplo
1. Argumentos de linha de comando	`java -jar app.jar --server.port=9090`
2. Variáveis de ambiente	`SERVER_PORT=9090`
3. `application-{profile}.properties`	`application-prod.properties`
4. `application.properties`	`server.port=8080`
5. Valores padrão do Spring	porta 8080

Relaxed binding: variáveis de ambiente sem `${}`

O Spring converte nomes de variáveis de ambiente para propriedades automaticamente: troca `_` por `.`, ignora maiúsculas e hífens. Assim, `SERVER_PORT` preenche `server.port` e `APP_CORS_ALLOWEDORIGINS` preenche `app.cors.allowed-origins`, mesmo sem escrever `${...}` no arquivo.

Variável de ambiente	Propriedade preenchida
`SERVER_PORT`	`server.port`
`SPRING_DATASOURCE_URL`	`spring.datasource.url`
`SPRING_PROFILES_ACTIVE`	`spring.profiles.active`
`APP_MONGODB_DATABASE`	`app.mongodb.database`

Aprofundando: dependências e configuração externa

Propriedades como record, validadas na inicialização

```
@Validated
@ConfigurationProperties(prefix = "app.jwt")
public record JwtProperties(
        @NotBlank String secret,
        @Positive long expirationMs) { }

// na classe principal ou em uma @Configuration:
@EnableConfigurationProperties(JwtProperties.class)
```

Aprofundando: dependências e configuração externa

Com `@Validated`, se `app.jwt.secret` estiver vazio a aplicação **nem sobe** e o log diz exatamente qual propriedade falta. É muito melhor descobrir isso na inicialização do que com um erro estranho no primeiro login.

06

Aprofundando: dependências e configuração externa

Aula 03 · Back-end

application.properties x application.yml

properties

- `spring.datasource.url=...`
- `spring.datasource.username=...`
- Uma chave completa por linha; fácil de copiar

vs

yml

- `spring:` → `datasource:` → `url: ...`
- Hierarquia por indentação (2 espaços)
- Um espaço errado quebra o arquivo

Aprofundando: dependências e configuração externa

Dica

No VS Code, com o *Spring Boot Extension Pack*, digite `spring.` no `application.properties` e use `Ctrl+Espaço`: o editor lista todas as propriedades válidas com descrição e valor padrão.

07

Quiz de revisão

Pergunta

Por que as dependências do Spring no `pom.xml` não têm ``?

A. Esqueceram de colocar

B. O *parent* do Spring Boot gerencia as versões compatíveis

C. O Maven ignora versões

D. Só funciona no VS Code

Resposta

Por que as dependências do Spring no `pom.xml` não têm ``?

A. Esqueceram de colocar

✓ O *parent* do Spring Boot gerencia as versões compatíveis

C. O Maven ignora versões

D. Só funciona no VS Code

Por quê: O `spring-boot-starter-parent` define versões testadas em conjunto.

Pergunta

Qual `scope` é usado no driver do PostgreSQL?

A test

B runtime

C provided

D system

Resposta

Qual `scope` é usado no driver do PostgreSQL?

A test

✓ runtime

C provided

D system

Por quê: O código não importa classes do driver; ele só é necessário na execução.

Pergunta

Em `server.port=${PORT:8080}`, o que acontece no Render?

A Sempre usa 8080

B Usa a variável PORT definida pelo Render

C Dá erro de sintaxe

D Usa a porta 80

Resposta

Em `server.port=${PORT:8080}`, o que acontece no Render?

A. Sempre usa 8080

✓ Usa a variável PORT definida pelo Render

C. Dá erro de sintaxe

D. Usa a porta 80

Por quê: Se a variável PORT existir, ela vence; senão vale o padrão 8080.

Pergunta

Qual anotação lê um grupo de propriedades com o mesmo prefixo?

A @Value

B @ConfigurationProperties

C @Bean

D @Profile

Resposta

Qual anotação lê um grupo de propriedades com o mesmo prefixo?

A @Value

✓ @ConfigurationProperties

C @Bean

D @Profile

Por quê: Ela preenche os atributos de uma classe a partir do prefixo, como `app.cors`.

Pergunta

Qual a precedência correta (maior primeiro)?

A application.properties > variável de ambiente

B Variável de ambiente > application.properties

C Os dois têm o mesmo peso

D Só o application.properties é lido

Resposta

Qual a precedência correta (maior primeiro)?

A application.properties > variável de ambiente

✓ Variável de ambiente > application.properties

C Os dois têm o mesmo peso

D Só o application.properties é lido

Por quê: Variáveis de ambiente sobrescrevem o arquivo, por isso o deploy funciona sem editar código.

Fechamento

Resumo da aula

- ✓ O `pom.xml` declara **o que** o projeto usa; o *parent* fixa versões compatíveis de tudo.
- ✓ `25` precisa bater com o JDK e com a imagem Docker.
- ✓ *Starters* agrupam dependências; `scope` define onde cada uma está disponível.
- ✓ O `application.properties` define **como** a aplicação se comporta; use `${VAR:padrao}` para o deploy.
- ✓ Leia valores com `@Value` (um) ou `@ConfigurationProperties` (grupo).

Exercícios

1. Adicione o starter de Validation ao seu projeto e confirme que o `import jakarta.validation.constraints.NotBlank` compila.

2. Mude a porta para `9090` usando só uma variável de ambiente (`PORT=9090 ./mvnw spring-boot:run`).

3. Crie `app.aplicacao.nome` e `app.aplicacao.versao` no `application.properties`, uma classe `AplicacaoProperties` e um endpoint que devolve os dois valores.

4. Crie os profiles `dev` e `prod` com níveis de log diferentes e alterne entre eles.

Para ir além

- docs.spring.io/spring-boot/appendix/application-properties: lista completa de propriedades
- maven.apache.org/pom.html: referência do POM

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 03?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web